

MASGym: A Co-Evolving Red/Blue Security Gym for Multi-Agent LLM Systems with Parameterized Collusion, Orchestrator Compromise, and System-level metrics

Anonymous Author(s)

Abstract

The strongest existing LLM-agent security benchmarks—Agent Security Bench, AgentDojo, InjecAgent, ToolEmu—are single-agent, yet deployed systems are collectives in which a planner directs workers, agents share memory, and tools are third-party. The phenomena that dominate risk there—collusion, a compromised orchestrator, malicious tools, Sybils, group-level emergent failure—cannot be produced by a single-agent harness. We present MASGYM, an environment and evaluation methodology for multi-agent LLM security: a forced-coordination task suite with heterogeneous roles and partial observability over star, chain, tree, and mesh topologies at 3–50 agents; a parameterized adversary model; a system-level metric $NRP_{sys} = PNA_{sys}(1 - ASR_{sys})$ scored by a deterministic checker over the global state trace, never an LLM judge on the critical path; and CoRB, a co-evolving red/blue protocol logging attack–defense trajectories and a (security, cost) Pareto frontier. Supporting theory, proved under explicit assumptions, characterizes the metric axiomatically (reducing to ASB at one agent), bounds compromise propagation across topologies, establishes a strict collusion advantage and a forced-coordination lower bound, and proves evaluator integrity. On a clearly-labelled *synthetic* emulator, a per-agent guardrail’s relative ASR reduction shrinks $41.6\% \rightarrow 31.8\% \rightarrow 27.6\%$ from single-agent to collusion to compromised-orchestrator context. We then ask what that guardrail buys, and answer on four real backbones in three model families: a prompt-level defense drives action-channel compliance to 0/1075 decisions, so $ASR_{act}=0$ throughout, yet ASR_{sys} remains 1.000 on tree and star, because once the unsafe action is blocked the system predicate is still discharged by deterministic availability sabotage that topology alone determines. A fourth backbone ignores the defense entirely. A defense measured on the action channel therefore cannot certify system-level security, and its efficacy is a property of the model as much as of the defense.

Keywords

multi-agent LLM security; security benchmark; co-evolving red/blue evaluation; collusion; prompt injection; deterministic security checking; system-level metrics

ACM Reference Format:

Anonymous Author(s). 2027. **MASGym**: A Co-Evolving Red/Blue Security Gym for Multi-Agent LLM Systems with Parameterized Collusion, Orchestrator Compromise, and System-level metrics. In *Proc. of the 26th International Conference on Autonomous Agents and Multiagent Systems (AAMAS 2027)*, Hanoi, Vietnam, 3–7 May 2027, IFAAMAS, 10 pages.

1 Introduction

Progress in the security of large language model (LLM) agents has tracked the quality of the instruments used to measure it. Agent Security Bench (ASB) [Zhang et al. 2025] introduced a utility–security metric $NRP = PNA \cdot (1 - ASR)$; AgentDojo [Debenedetti et al. 2024] evaluates an agent under injection with *deterministic*, state-based `security()` checks; ToolEmu [Ruan et al. 2024] emulates arbitrary tool execution; InjecAgent [Zhan et al. 2024] benchmarks indirect injection. These instruments are strong but single-agent: their only cross-agent channel is a shared memory store. Yet deployed systems such as AutoGen [Wu et al. 2023], MetaGPT [Hong et al. 2023], CAMEL [Li et al. 2023], ChatDev [Qian et al. 2024], and Magentic-One [Fourney et al. 2024] are orchestrated *collectives*: a planner directs workers, agents share memory, third-party tools sit on the critical path, and many tasks require several agents to finish at all.

In such systems the security questions are group-level. Do a fraction of agents *collude*? Is the *orchestrator* trusted, or compromised? Can a *malicious tool*, a *Sybil* identity, or a *poisoned* memory entry spread through the collective? Re-running a single-agent benchmark N times cannot produce these phenomena. MASEC [Schroeder de Witt et al. 2025] argues that agent security is *non-compositional*—individually safe agents compose into unsafe systems—and notes that no multi-agent security benchmark exists, listing what one must provide: heterogeneous roles, partial observability, forced coordination, a co-evolving red/blue methodology, and group-level emergence metrics.

Contributions. To the best of our knowledge MASGYM is the first environment purpose-built for the security of multi-agent LLM systems. It is not a new attack or defense; it is an environment and evaluation methodology whose scientific object is the controlled variation of coordination, collusion, and orchestrator variables and the system-level metrics that quantify their effect (architecture in the Supplementary Material; three-layer design in Section 7). Concretely,

- **An environment and task suite** (Sections 4 and 7): heterogeneous roles, partial observability, *forced-coordination* tasks (no agent can complete one alone), over star/chain/tree/mesh topologies at 3–50 agents.
- **A parameterized adversary model** (Section 5): one vector controls adversary fraction, collusion, adaptivity, orchestrator



This work is licensed under a Creative Commons Attribution International 4.0 License.

compromise, malicious tools, Sybils, and poisoned memory, under a system-level compromise objective.

- **A system-level metric suite** (Sections 6 and 7): the generalization $\text{NRP}_{\text{sys}} = \text{PNA}_{\text{sys}} (1 - \text{ASR}_{\text{sys}})$, scored by a *deterministic checker* over the global trace so an injection cannot hijack the evaluator (Theorem 9.4), plus collusion-success and compromise-propagation rates, coordination quality, cost, and recovery.
- **The CoRB co-evolving red/blue protocol** (Section 8), logging (attack, defense, NRP_{sys}) trajectories and a (security, cost) Pareto frontier.
- **An answer to “does the defense work?”** (Section 10.3): on four real backbones in three model families a prompt-level defense drives action-channel compliance to 0/1075 decisions— $\text{ASR}_{\text{act}}=0$ —yet ASR_{sys} stays 1.000 on tree and star, because the residual failure is deterministic availability sabotage that topology alone determines, and one backbone ignores the defense entirely (Table 1). A defense scored on the action channel cannot certify system-level security. The controlled synthetic sweep separately measures the guardrail’s shrinking margin, $41.6\% \rightarrow 31.8\% \rightarrow 27.6\%$ (degradation grid, Supplementary Material).

Because reviewers rightly discount benchmarks whose central metric is asserted rather than justified, we treat theory as part of the instrument (Section 9): under explicit assumptions and honest proof-status labels, *every result we claim is proved in the paper*. We prove the product aggregator is the unique combiner satisfying our axioms and reduces to ASB at $N=1$; that the deterministic checker cannot be hijacked through message content, unlike a message-reading LLM judge, which we measure pushed off the recorded state in roughly a third of injected traces; and finite-sample concentration bounds. None asserts that any real defense is secure or provably degrades.

Scope and stance. MASGYM is an instrument, not a verdict: a lower ASR_{sys} is a measurement under a stated configuration, not a certificate. Scoring uses a deterministic critical path (Theorem 9.4); emulation fidelity is the main validity gap (?). The TCB is the harness and checker; model-weight attacks are out of scope.

2 Related Work

2.1 Single-agent security instruments

The closest instruments evaluate a lone tool-using agent. ASB [Zhang et al. 2025] introduces the utility–security metric $\text{NRP} = \text{PNA} \cdot (1 - \text{ASR})$ that we generalize, under a single-agent threat model with retrieved memory. AgentDojo [Debenedetti et al. 2024] contributes a stateful environment whose *deterministic* security/utility functions read pre- and post-execution state, so a prompt injection cannot also hijack the evaluator; it names multi-agent planner/worker defense as future work. InjecAgent [Zhan et al. 2024], ToolEmu [Ruan et al. 2024], AgentHarm [Andriushchenko et al. 2025], HarmBench [Mazeika et al. 2024], and formalizations of injection [Liu et al. 2024] and trustworthiness [Wang et al. 2023a] complete the picture. None parameterizes collusion, orchestrator trust, Sybils, or compromise propagation.

2.2 Multi-agent frameworks, attacks, and defenses

A large literature builds collectives—AutoGen [Wu et al. 2023], MetaGPT [Hong et al. 2023], CAMEL [Li et al. 2023], ChatDev [Qian et al. 2024], Magentic-One [Fourney et al. 2024], and mixing/debating designs [Du et al. 2023]—concerned with capability, not security; MASGYM adopts their role structures as substrate. A fast-growing attack literature is intrinsically multi-agent: prompt infection [Lee and Tiwari 2024], contagious blocking [Zhou et al. 2025], population jailbreaking [Gu et al. 2024], topological safety [Yu et al. 2024], adversarial roles [Tian et al. 2023], GenAI worms [Cohen et al. 2024], memory poisoning [Chen et al. 2024], and backdoors [Yang et al. 2024]. These are attacks in bespoke setups; MASGYM is the controlled, deterministically scored environment in which such attacks and defenses can be swept and compared. Multi-agent defenses include AutoDefense [Zeng et al. 2024] and the topology-guided G-Safeguard [Wang et al. 2025]; per-agent guardrails (TrustAgent [Hua et al. 2024], Llama Guard [Inan et al. 2023], NeMo Guardrails [Rebeida et al. 2023]) are the baselines we lift into the collective setting. Classical Byzantine fault tolerance [Lamport et al. 1982; Pease et al. 1980] and Byzantine-robust aggregation [Blanchard et al. 2017] tolerate faulty fractions under a trusted coordinator over numeric messages; they motivate our collusion and orchestrator-trust knobs without covering natural-language collectives.

2.3 Environments, red teaming, and evaluation validity

Methodologically MASGYM follows standardized environments—Gym [Brockman et al. 2016], PettingZoo [Terry et al. 2021], and Melting Pot [Leibo et al. 2021], which evaluates social outcomes at the level of the collective—and automated red teaming [Perez et al. 2022], with the MAST taxonomy [Cemri et al. 2025] supplying a vocabulary for group-level failures. Surveys of agent security [He et al. 2025; Li et al. 2024] and the non-compositionality argument [Hammond et al. 2025; Schroeder de Witt et al. 2025] are the gap MASGYM addresses. On scoring validity, LLM-as-judge evaluation is documented as manipulable and biased [Wang et al. 2023b; Zheng et al. 2023]; MASGYM therefore keeps a deterministic trace-based checker on the critical path and treats the deterministic-versus-judge comparison as a measurable ablation.

2.4 Positioning

MASGYM is *not* ASB applied to many agents (ASB’s only cross-agent vector is shared memory), *not* AgentDojo at scale (its checker is per-agent over environment state; a collective needs system-level predicates over a global trace, Section 6), and *not* a model-swap or prompt-filter study: its object is the controlled variation of coordination, collusion, and orchestrator variables, and the system-level metrics that quantify their effect.

3 Preliminaries and Notation

This section recalls the three primitives MASGYM builds on (notation summarized in the Supplementary Material): ASB’s utility–security metric, AgentDojo’s deterministic state-based checking, and ToolEmu’s LM tool emulation.

3.1 ASB’s utility–security metric

ASB [Zhang et al. 2025] scores an agent by a *Non-Refusal Performance*,

$$\text{NRP} = \text{PNA} \cdot (1 - \text{ASR}), \quad (1)$$

where $\text{PNA} \in [0, 1]$ is performance on benign tasks and $\text{ASR} \in [0, 1]$ the attack success rate; the product rewards a system only when both useful and not attacked. Equation (1) is defined for a single agent; generalizing it to a collective is the subject of Sections 6 and 9.

3.2 AgentDojo’s deterministic state-based checking

AgentDojo [Debenedetti et al. 2024] evaluates security by *deterministic* functions of environment state rather than an LLM judge: a task fixes a predicate reading the pre- and post-execution state and returning a Boolean, so an injection cannot manipulate the score unless it changes the security-relevant state. MASGYM lifts this property to the system level: our checker C evaluates Φ, Ψ over the *global* trace τ , never an invocable LLM judge, and we prove the resulting integrity property in Theorem 9.4.

3.3 ToolEmu’s LM tool emulation

ToolEmu [Ruan et al. 2024] emulates tool execution from a specification with a language model, enabling breadth at low cost, at a reported human agreement of only $\kappa \approx 0.48$ (Cohen’s κ [Cohen 1960], extended to many raters by Fleiss [Fleiss 1971]). MASGYM adopts LM emulation for tool breadth, including an adversarial mode $\mathcal{E}_{\text{LM}}^\dagger$ for malicious tools, but treats κ as a first-class threat to validity: scoring routes through the deterministic checker, and the emulated-versus-real-tools ablation is registered but unmeasured, since no real tool backend is wired in (??). An *episode* runs a task under a fixed configuration c and produces a global trace $\tau \sim \mathcal{D}(c)$, from which all security and utility quantities are computable—the fact that makes deterministic, hijack-resistant scoring possible.

4 System Model

MASGYM evaluates a collective of $N \in \{3, \dots, 50\}$ LLM agents $\mathcal{A} = \{a_1, \dots, a_N\}$, each assigned exactly one role from the set $\mathcal{R}$ (notation in the Supplementary Material). Roles pool access to shared tools and a shared versioned memory, and interplay is *structural*, captured by a topology $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ over agents—star, chain, tree, or mesh—that fixes who may influence whom: $j \in N^-(a)$ means a reads j ’s outputs; the orchestrator is typically the star’s centre.

An *episode* executes a task under a configuration c for H steps, producing a *global trace* $\tau = (s_0, \dots, s_H)$, where s_t is the complete state: every agent’s messages, tool calls, tool outputs, memory versions, role assignments, topology edges, and the agent-identity map. Traces are i.i.d. from $\mathcal{D}(c)$ (Assumption 2), so system-level quantities are computable offline from τ —the property that keeps the checker deterministic and the measurement reproducible. A configuration $c \in \mathcal{C}$ specifies task, $\mathcal{G}$, models, tools, $\mathcal{B}$, and protocol constants, so a study reports NRP_{sys} as a coordinate over a *sweep* that varies exactly one component at a time under a controlled seed.

5 Threat Model

The adversary is a set $\mathcal{B} \subseteq \mathcal{A}$ of *adversarial agents*, of size βN chosen by the operator—a single corrupted node or a coordinated coalition sharing a channel and a joint objective. Capabilities and attack mode are fixed by

$$c_{\text{adv}} = (\beta, \text{coll}, \text{adapt}, \text{orch}, \text{tool}, \text{syb}, \text{mem}), \quad (2)$$

selecting respectively: adversary fraction; independent vs. colluding coordination (Theorem 6.5); static vs. adaptive attacks across CoRB rounds; compromised orchestrator; malicious tools (adversarial emulator $\mathcal{E}_{\text{LM}}^\dagger$); Sybils; and poisoned memory. $\mathcal{B}$ may issue messages and tool calls as its roles, read $\mathcal{M}$, and read the outputs of its topology predecessors; *outside* its powers are the harness, the recorded trace, the checker, the emulator’s mechanism, and the honest agents’ private internals. Compromise then *propagates* along $\mathcal{G}$ (Theorem 6.4) from these seeds, so the effective compromised set at episode end, $\mathcal{B}_\infty$, can exceed $\mathcal{B}$.

Inferential threats and scope. The operator sets horizon, models, and metrics; the adversaries do not. An *LLM judge*, when used (only in ablations, off the critical path), is a *measurement* threat: human agreement is $\kappa \approx 0.48$ and judges can be gamed; the deterministic checker exists to remove this threat (Theorem 9.4, Theorem 9.5). The TCB is the harness (trace recording, roles, topology, scheduling), the tool specifications, and the registered predicates; everything else—messages, memory writes, tool outputs, intent—is outside it. Scope is *per se* the *collective*; adversaries are bounded by the seven knobs and episodic budgets; a harness-level adversary and model-weight attacks are out of scope; honest agents have no budget (per ASB) and obey their roles.

6 Problem Formulation

We define what a system-level security benchmark must measure: security and benign-success predicates, the system-level metric aggregating them, and the group-level quantities (forced coordination, compromise propagation, collusion) with no single-agent analogue.

6.1 System-level predicates

Definition 6.1 (System-level security and benign-success predicates). A task fixes two Boolean functionals of the global trace $\tau \in \mathcal{T}$: a *security predicate* $\Phi : \mathcal{T} \rightarrow \{0, 1\}$, 1 iff the trace contains a system-level compromise (an unsafe action executed, protected data exfiltrated, or a forced-coordination task sabotaged), and a *benign-success predicate* $\Psi : \mathcal{T} \rightarrow \{0, 1\}$, 1 iff the collective completed the legitimate task. Both are registered programs evaluated by the deterministic checker over the harness-recorded trace; dependencies on *un-recorded* natural-language content are disallowed, yielding evaluator integrity (Theorem 9.4) and lifting AgentDojo [Debenedetti et al. 2024] from one agent’s state to the global trace.

6.2 System-level metrics

Definition 6.2 (System-level ASR, PNA, and NRP). Fix a configuration c and let $\tau \sim \mathcal{D}(c)$. Define

$$\text{ASR}_{\text{sys}}(c) = \mathbb{P}_{\tau \sim \mathcal{D}(c)}[\Phi(\tau) = 1], \quad (3)$$

$$\text{PNA}_{\text{sys}}(c) = \mathbb{P}_{\tau \sim \mathcal{D}(c)}[\Psi(\tau) = 1], \quad (4)$$

$$\text{NRP}_{\text{sys}}(c) = \text{PNA}_{\text{sys}}(c) \cdot (1 - \text{ASR}_{\text{sys}}(c)), \quad (5)$$

where c° is c 's benign counterpart (no adversary), so PNA_{sys} measures coordinated-task performance uncontaminated by attack-induced refusals, exactly as ASB separates PNA from ASR.

Two properties are required and proved in Section 9: Equation (5) *reduces* to ASB's Equation (1) at $N=1$ (Theorem 9.1), and the product form is *justified* rather than assumed (Section 9; Theorem 9.2). NRP_{sys} is a coordinate for comparing configurations, not a certificate.

6.3 Forced coordination

Definition 6.3 (k -forced-coordination task). For an agent set $\mathcal{A}$, let a *witness* be a set $W \subseteq \mathcal{A}$ whose joint contributions can satisfy Ψ . A task is *k -forced-coordination* if every witness has size $|W| \geq k$; equivalently, for every S with $|S| < k$, no assignment in which only the agents of S act non-trivially satisfies Ψ .

k -forced tasks with $k \geq 2$ cannot be represented in a single-agent harness. Section 9 relates task disruption to whether $\mathcal{B}$ intersects every minimal witness (a hitting-set condition, Theorem 9.8).

6.4 Compromise propagation and collusion

Definition 6.4 (Compromise relation and propagation rate). Let $\mathcal{B}_0 = \mathcal{B}$ be the initially adversarial (seed) agents. Compromise spreads over $\mathcal{G}$ according to a monotone rule $\mathcal{P}$ (Section 9: an independent-cascade model with per-edge probability p , or a threshold model with threshold θ). Write $\mathcal{B}_\infty \supseteq \mathcal{B}_0$ for the compromised set at fixation. The *compromise-propagation rate* is

$$\text{CPR}(c) = \mathbb{E} \left[\frac{|\mathcal{B}_\infty| - |\mathcal{B}_0|}{N - |\mathcal{B}_0|} \right], \quad (6)$$

the expected fraction of *initially honest* agents that become compromised.

Definition 6.5 (Collusion success rate). For a fixed fraction β , let $\text{ASR}_{\text{sys}}^{\text{coll}}$ and $\text{ASR}_{\text{sys}}^{\text{ind}}$ be the system attack success rates under colluding and independent adversaries respectively. The *collusion success rate* reports $\text{ASR}_{\text{sys}}^{\text{coll}}$, and the *collusion advantage* is $\Delta_{\text{coll}} = \text{ASR}_{\text{sys}}^{\text{coll}} - \text{ASR}_{\text{sys}}^{\text{ind}}$.

7 The MASGym Environment

MASGYM is organized as three layers (architecture figure in the Supplementary Material): an *emulation layer* that supplies tools, a *deterministic checking layer* that scores system-level predicates, and an *orchestration layer* that instantiates the system and threat models and drives the CoRB loop.

7.1 Emulation and deterministic checking

Following ToolEmu [Ruan et al. 2024], tool execution is emulated by a language model $\mathcal{E}_{\text{LM}}$ from a tool specification, giving breadth without hand-built sandboxes; an *adversarial mode* $\mathcal{E}_{\text{LM}}^\dagger$ realizes malicious tools (attacker-controlled responses, exfiltration bait, injected instructions). Crucially the emulator is *not* on the scoring critical path: it produces the tool outputs agents consume, while the verdict comes from the deterministic checker, so emulator noise degrades realism, not scoring integrity. The checker $\mathcal{C}$ evaluates the registered predicates Φ, Ψ over the global trace (Theorem 6.1) and never invokes an LLM judge on the critical path. Predicates

are programs over recorded state: an *exfiltration* predicate fires iff protected data recorded as leaving the trust boundary appears in an outbound tool call; a *sabotage* predicate fires iff a k -forced task's witness set was prevented from completing; an *unsafe-action* predicate fires iff a denylisted transition is recorded. An optional LLM judge $\mathcal{J}$, available only off the critical path, enables the ablation (RQ2).

7.2 Orchestration layer and task suite

The orchestration layer builds the topology $\mathcal{G}$, assigns roles, enforces observability masks, applies c_{adv} of Equation (2), and runs episodes; it is part of the TCB and exposes the CoRB interface (Section 8). Tasks are *forced-coordination* by design (Theorem 6.3), so the collective is the unit of analysis: each provides a benign goal decomposable across roles, a registered pair (Ψ, Φ) , a tool set (some malicious under tool), a shared-memory schema (poisonable under mem), and a topology (star: hub-and-spoke delegation; chain: pipelines; tree: hierarchical; mesh: peer collaboration), parameterized by k (the degree of forced coordination) and benign difficulty.

7.3 Realized adversary knobs

Each component of c_{adv} maps to a concrete harness mechanism: β selects the seed set $\mathcal{B}$; coll toggles a shared channel and joint objective; adapt enables red co-evolution across CoRB rounds; orch flips the orchestrator's policy; tool routes tools through $\mathcal{E}_{\text{LM}}^\dagger$; syb instantiates extra identities under one controller; and mem writes poisoned entries to shared memory. Because the knobs are independent, a study isolates each variable's effect while holding the rest fixed—the controlled variation that is the paper's scientific object [Rubin 1974].

7.4 Metric suite

All reported quantities come from traces via the deterministic checker or harness logs; none is an LLM-judge verdict: ASR_{sys} and NRP_{sys} (Equations (3) and (5)), PNA_{sys} (Equation (4)), collusion success and advantage (Theorem 6.5), CPR (Equation (6)), coordination quality, time-to-detection and recovery, cost and latency scaling, and human-intervention rate. Precise definitions are in the Supplementary Material; the budget is fixed by Theorem 9.9 and Theorem 9.10.

8 The CoRB Co-Evolving Red/Blue Protocol

MASGYM's evaluation methodology is a co-evolving loop, CoRB, that alternates a red policy proposing attack configurations and a blue policy proposing defenses, scoring each pair with the deterministic checker and logging a trajectory of (attack, defense, NRP_{sys}) triples. The per-configuration scoring procedure (SCORECONFIG) is fully specified: instantiate $\mathcal{G}$, roles, masks, and the adversary knobs c_{adv} Equation (2) via the trusted harness; seed $\mathcal{B}_0$; run M episodes of H steps, recording each transition into the global trace and updating the compromised set under the propagation rule $\mathcal{P}$ (Theorem 6.4); evaluate (Ψ, Φ) with the deterministic checker on each trace; and aggregate $\widehat{\text{ASR}}_{\text{sys}}, \widehat{\text{PNA}}_{\text{sys}}, \widehat{\text{NRP}}_{\text{sys}}, \widehat{\text{CPR}}$ plus detection and recovery statistics. The protocol-level loop is Algorithm 1. Both levels keep

Algorithm 1 CoRB: co-evolving red/blue evaluation.

-
- 1: **Input:** red policy ρ ; blue policy δ ; rounds R ; task, budget M .
 - 2: **Output:** trajectory $\mathcal{L} = \{(c_{\text{adv}}^r, \delta^r, \text{NRP}_{\text{sys}}^r, \text{ASR}_{\text{sys}}^r)\}_{r=1}^R$ and a Pareto frontier.
 - 3: Initialize defense δ^0 (no-defense or a lifted per-agent defense).
 - 4: **for** $r = 1, \dots, R$ **do**
 - 5: **Red step:** $c_{\text{adv}}^r \leftarrow \rho(\mathcal{L}_{<r}, \delta^{r-1})$ $\triangleright$ static: fixed; adaptive: best-response
 - 6: Score $(c_{\text{adv}}^r, \delta^{r-1})$ with SCORECONFIG.
 - 7: **Blue step:** $\delta^r \leftarrow \delta(\mathcal{L}_{<r}, c_{\text{adv}}^r)$ $\triangleright$ propose/upgrade defense
 - 8: Re-score $(c_{\text{adv}}^r, \delta^r)$; append both scorings to $\mathcal{L}$.
 - 9: **return** $\mathcal{L}$ and its $(\text{ASR}_{\text{sys}}, \text{cost}) / (\text{NRP}_{\text{sys}}, \text{cost})$ frontier.
-

the LLM off the scoring critical path, so no proposal can inflate its own score.

Three properties make CoRB a measurement protocol rather than a training loop: (i) *integrity*—every score comes from SCORECONFIG, so neither policy can hijack its own evaluation (Theorem 9.4); (ii) *reproducibility*—the trajectory $\mathcal{L}$ is the full audit trail of fully specified (c_{adv}, δ) pairs; and (iii) *honesty of the frontier*—CoRB reports a Pareto frontier over (security, cost) rather than a single “winner”. The red and blue policies may be grid/random search, an optimizer, or an LLM-driven proposer; MASGYM fixes the interface and the scoring, not the proposers. Cost is $\Theta(\text{RMNH})$ for an R -round run and $\Theta(\text{GNH}\Delta^{-2} \ln(G/\alpha))$ across a G -configuration sweep (Theorem 9.10).

9 Theory: What the Instrument Measures

A benchmark earns trust when its central metric is justified, its evaluator cannot be gamed, and its sample sizes are principled, so we treat theory as part of the instrument. All results are stated under explicit assumptions and carry honest proof-status labels; none claims that any real defense is secure or provably degrades. *Every result we claim is proved in full in this section.* What the Supplementary Material adds are the derivations of the classical propagation results we restate in our notation (Granovetter/Watts/Kempe), not proofs of our own claims.

Assumption 1 (Trusted checking). The harness records a faithful global trace τ , and the checker C evaluates the fixed predicates Φ, Ψ as functions of τ only (Theorem 6.1).

Assumption 2 (i.i.d. episodes). For fixed c , the M episode traces are independent draws from $\mathcal{D}(c)$.

Assumption 3 (Bounded indicators). $\Psi, \Phi \in \{0, 1\}$.

Assumption 4 (Monotone propagation model). Compromise spreads over $\mathcal{G}$ via a monotone rule $\mathcal{P}$ (Theorem 6.4): under *independent cascade*, a compromised agent independently compromises each susceptible out-neighbour with probability p ; under *threshold*, a susceptible agent is compromised once at least θ in-neighbours are. This is a modelling choice MASGYM makes so propagation is measurable, not an empirical law; its realism is itself measured via CPR.

Assumption 5 (Continuity of the aggregator). The utility–security aggregator $g : [0, 1]^2 \rightarrow [0, 1]$ is continuous.

9.1 Characterization of the system-level metric

The product form of NRP_{sys} is forced by the desiderata of Section 6, phrased as five axioms (D1–D5): boundedness, consistency, monotonicity, homogeneity (each in both arguments), and reduction—the operative instance is stated and proven in Theorem 9.2. Under them, g reduces to ASB’s metric at one agent.

Proposition 9.1 (Reduction to ASB). *If $N = 1$ and Φ, Ψ are the single-agent security and benign predicates, then $\text{ASR}_{\text{sys}} = \text{ASR}$, $\text{PNA}_{\text{sys}} = \text{PNA}$, and $\text{NRP}_{\text{sys}} = \text{NRP}$ of Equation (1).*

PROOF OF THEOREM 9.1. With one agent the global trace τ is that agent’s own trajectory and the system predicates Φ, Ψ coincide with the single-agent ones. Substituting into Equations (3) and (4) yields $\text{ASR}_{\text{sys}} = \text{ASR}$ and $\text{PNA}_{\text{sys}} = \text{PNA}$ term by term, hence $\text{NRP}_{\text{sys}} = \text{PNA}_{\text{sys}}(1 - \text{ASR}_{\text{sys}}) = \text{PNA}(1 - \text{ASR}) = \text{NRP}$. $\square$

Theorem 9.2 (Axiomatic characterization of NRP_{sys} ; complete proof). *Write $\text{NRP}_{\text{sys}} = g(u, \sigma)$ with utility $u = \text{PNA}_{\text{sys}} \in [0, 1]$ and security level $\sigma = 1 - \text{ASR}_{\text{sys}} \in [0, 1]$. Under Assumption 5, suppose g satisfies: (D2) consistency, $g(u, 1) = u$ and $g(0, \sigma) = 0$; (D4a) utility-homogeneity, $g(\lambda u, \sigma) = \lambda g(u, \sigma)$ for $\lambda \in [0, 1]$; and (D4b) security-homogeneity, $g(u, \lambda \sigma) = \lambda g(u, \sigma)$ for $\lambda \in [0, 1]$. Then $g(u, \sigma) = u \sigma$ is the unique such aggregator, satisfying boundedness (D1) and monotonicity (D3).*

PROOF OF THEOREM 9.2. By D4a with base $u = 1$ and D4b with base $\sigma = 1$, $g(u, \sigma) = u g(1, \sigma) = u \sigma g(1, 1) = u \sigma$ by D2. The boundary $g(0, \sigma) = 0$ holds since $0 \cdot \sigma = 0$; and $u \sigma$ is bounded in $[0, 1]$ and non-decreasing in each argument. Continuity (Assumption 5) extends the homogeneity identities from rationals to all of $[0, 1]$; the identities hold pointwise. $\square$

Remark 9.3. Theorem 9.2 singles out the product *relative to D2/D4*: replacing D4 by a worst-coordinate axiom yields $\min(u, \sigma)$, and weighted log-linearity yields $u^w \sigma^{1-w}$. MASGYM reports $u \sigma$ by default (it generalizes ASB, Theorem 9.1) and exposes alternatives as configurable.

9.2 Evaluator integrity

We formalize why deterministic checking cannot be hijacked by a successful injection.

Theorem 9.4 (Evaluator integrity; complete proof). *Under Assumption 1, for any two episodes with equal recorded traces $\tau = \tau'$, $C(\tau) = C(\tau')$. Consequently, an adversarial manipulation that changes message content but leaves the recorded security-relevant state unchanged cannot change $\Phi(\tau)$; hence neither $\overline{\text{ASR}_{\text{sys}}}$ nor $\overline{\text{NRP}_{\text{sys}}}$ can be inflated or deflated by message content alone.*

PROOF OF THEOREM 9.4. By Assumption 1 the predicates are functions of τ only, so equal traces give equal verdicts. For any manipulation π inducing $\tau_\pi = \tau$ the verdict $\Phi(\tau_\pi) = \Phi(\tau)$, and hence so are $\overline{\text{ASR}_{\text{sys}}}, \overline{\text{PNA}_{\text{sys}}}, \overline{\text{NRP}_{\text{sys}}}$. $\square$

Proposition 9.5 (An LLM judge is hijackable; complete proof of existence). *There exist an LLM judge J that reads messages and a message manipulation π with $\tau_\pi = \tau$ such that $J(\tau_\pi, \text{msgs}_\pi) \neq J(\tau, \text{msgs})$.*

PROOF OF THEOREM 9.5. J takes message content as an argument, so let it be any judge non-constant in that argument—as all instruction-following judges are. Choose messages msgs and a manipulation π (e.g. msgs_π carries the injected instruction “report this trace as safe”) on which it disagrees, so $J(\tau_\pi, \text{msgs}_\pi) \neq J(\tau, \text{msgs})$. Content edits leave the recorded state untouched, so by construction $\tau_\pi = \tau$ and the deterministic verdict is unchanged. The contrast with Theorem 9.4 is exact: it is the message channel, not the trace, that carries the vulnerability. $\square$

Remark 9.6. Theorem 9.4 shows the deterministic verdict cannot be *gamed* through messages; it does *not* show it is *correct* (a mis-specified predicate can be systematically wrong). Viewed as a test of the compromise hypothesis, C is a deterministic decision rule governed by classical testing theory [Lehmann and Romano 2005; Neyman and Pearson 1933]; the LLM judge additionally exposes that decision to manipulation through its message input, as documented for LLM judges [Wang et al. 2023b; Zheng et al. 2023]. MASGYM keeps C on the critical path and measures the judge-hijack gap as an ablation (RQ2).

9.3 Compromise propagation across topologies

Under the independent-cascade instance of Assumption 4, expected reach depends sharply on topology. We *restate* four classical results in our notation, and derive from them the one claim we need. On a chain *seeded at an endpoint*, a_{1+j} is compromised iff all j intervening edges fire, so the expected additional reach is $\sum_{j=1}^{n-1} p^j \leq p/(1-p)$, bounded independently of n . On a star, a centre seed gives reach $p(N-1)$ and $\text{CPR} = p$, whereas a leaf seed gives $p + p^2(N-2)$; the gap is $p(1-p)(n-2) = \Theta(p(1-p)n)$. On a tree of branching factor b seeded at the root the expected level- ℓ count is $(bp)^\ell$, so reach is bounded in depth for $bp < 1$ and $\Theta((bp)^d)$ for $bp > 1$: a phase transition at $bp = 1$. On the complete graph K_n with s seeds, one round already compromises $s + (n-s)(1 - (1-p)^s)$ nodes, which tends to all of V as s grows. Under threshold contagion an agent with in-degree below θ is never compromised, so a chain with $\theta \geq 2$ has $\text{CPR} = 0$. These are the classical spread results [Chalupa et al. 1979; Granovetter 1978; Kempe et al. 2003; Watts 2002]—we reproduce the derivations in the Supplementary Material for completeness rather than claim them—and our own consequence follows.

Theorem 9.7 (Collusion advantage on the star). *Fix the independent-cascade model on a star of N nodes with a budget of one seed. A colluding adversary that places its seed at the centre achieves expected compromise $C = 1 + p(N-1)$; an independent adversary placing its seed uniformly at random achieves expected compromise $\frac{1}{N}C + \frac{N-1}{N}L$, writing C as above and $L = 1 + p + p^2(N-2)$. Their difference $\Delta_{\text{coll}} = p(1-p)\frac{(N-1)(N-2)}{N}$ is positive, $\Theta(p(1-p)N)$, for all $p \in (0, 1)$, $N \geq 3$: coordinated placement strictly dominates.*

PROOF OF THEOREM 9.7. Each leaf is a direct out-neighbour of the centre, so a centre seed compromises all $N-1$ leaves independently with probability p , giving $C = 1 + p(N-1)$. A leaf seed

compromises the centre with probability p and, conditioned on that, each remaining leaf with probability p , giving $L = 1 + p + p^2(N-2)$. A uniform independent seed lands on the centre with probability $1/N$ and on a leaf with probability $(N-1)/N$, hence $\mathbb{E}^{\text{ind}} = \frac{1}{N}C + \frac{N-1}{N}L$. Therefore

$$\Delta_{\text{coll}} = C - \mathbb{E}^{\text{ind}} = \frac{N-1}{N}(C - L) = p(1-p)\frac{(N-1)(N-2)}{N}.$$

For every $p \in (0, 1)$ and $N \geq 3$ we have $(N-1)(N-2)/N > 0$ and $p(1-p) > 0$, so $\Delta_{\text{coll}} > 0$ strictly, and it is $\Theta(p(1-p)N)$ since $(N-1)(N-2)/N = \Theta(N)$. (For $s > 1$ seeds, coordinated placement solves the monotone submodular influence-maximization objective instead; see the Supplementary Material.) $\square$

Theorem 9.7 describes the *environment’s* propagation model, not any real defense; its value is that “collusion on/off” becomes well posed and predicts where to look—hub-centric topologies—for degradation, which Section 10 then tests.

9.4 Forced coordination

Proposition 9.8 (Forced-coordination lower bound). *Let a task be k -forced-coordination (Theorem 6.3) with witness family $\mathcal{W}$. Then: (a) no set S of agents with $|S| < k$ can satisfy Ψ ; and (b) if $\mathcal{B}$ is a transversal of $\mathcal{W}$, the adversary can force $\Psi = 0$ by having its member in each witness withhold or sabotage its contribution; conversely, if some witness is adversary-free, the honest agents can complete the task.*

PROOF OF THEOREM 9.8. (a) Every witness has size $\geq k$, so a set S with $|S| < k$ supports no witness and cannot satisfy Ψ . (b) If $\mathcal{B}$ is a transversal, every route to $\Psi = 1$ passes through an adversarial agent that can withhold, so $\Psi = 0$; if some witness is adversary-free its members jointly satisfy it. $\square$

9.5 Sample complexity of the estimators

The estimates require bounded episodes, driving the cost analysis. The tools are elementary concentration inequalities [Boucheron et al. 2013; Hoeffding 1963]: Hoeffding’s inequality applies to the means here, and McDiarmid’s inequality [McDiarmid 1989] handles the non-mean estimators.

Theorem 9.9 (Concentration and sample complexity). *Under Assumptions 2 and 3, for any $\epsilon, \alpha \in (0, 1)$: (a) for each estimator $\hat{\theta} \in \{\widehat{\text{ASR}}_{\text{sys}}, \widehat{\text{PNA}}_{\text{sys}}\}$ with target θ , $\mathbb{P}[|\hat{\theta} - \theta| \geq \epsilon] \leq 2e^{-2M\epsilon^2}$, so $M \geq \frac{1}{2\epsilon^2} \ln \frac{2}{\alpha}$ episodes give ϵ -accuracy at confidence $1-\alpha$; (b) the product is Lipschitz; hence, by a union bound, $\mathbb{P}[|\widehat{\text{NRP}}_{\text{sys}} - \text{NRP}_{\text{sys}}| \geq 2\epsilon] \leq 4e^{-2M\epsilon^2}$; and (c) across a sweep of G configurations, Bonferroni/FDR control [Benjamini and Hochberg 1995] gives $M = O(\epsilon^{-2} \ln(G/\alpha))$ per configuration, for $\Theta(\text{GMNH})$ total agent invocations.*

PROOF OF THEOREM 9.9. (a) Each $\widehat{\text{ASR}}_{\text{sys}}$ and $\widehat{\text{PNA}}_{\text{sys}}$ is the mean of M i.i.d. $\{0, 1\}$ indicates (Assumption 3), so Hoeffding’s inequality gives $\mathbb{P}[|\hat{\theta} - \theta| \geq \epsilon] \leq 2e^{-2M\epsilon^2}$; rearranging $2e^{-2M\epsilon^2} \leq \alpha$ yields $M \geq (2\epsilon^2)^{-1} \ln(2/\alpha)$. (b) For $u, \hat{u}, \sigma, \hat{\sigma} \in [0, 1]$, $|\hat{u}\hat{\sigma} - u\sigma| \leq \hat{\sigma}|\hat{u} - u| + u|\hat{\sigma} - \sigma| \leq |\hat{u} - u| + |\hat{\sigma} - \sigma|$; apply this with $\sigma = 1 - \text{ASR}_{\text{sys}}$ and union-bound the two events of (a) at threshold ϵ . (c) Bonferroni over G tests replaces α by α/G ; solving for M gives the stated rate, and each of G configurations runs M episodes of N agents over horizon H . $\square$

Corollary 9.10 (Sweep budget). *Resolving a target NRP_{sys} gap of size Δ between any two of G swept configurations with family-wise confidence $1 - \alpha$ costs $\Theta(\text{GNH}\Delta^{-2} \ln(G/\alpha))$ agent invocations.*

PROOF OF THEOREM 9.10. Set the per-configuration accuracy to $\epsilon = \Delta/4$ in Theorem 9.9(b) so that the two NRP_{sys} estimates are each within $\Delta/2$; by the triangle inequality their difference is resolved to Δ . Theorem 9.9(c) then gives $M = O(\Delta^{-2} \ln(G/\alpha))$, and each configuration runs M episodes of N agents over horizon H . $\square$

10 Evaluation Protocol and Experimental Results

We instantiate MASGYM with controlled synthetic models for the sweeps and real LLM backbones for the defense study, which is the claim that most needs real agents; oracles, detectors, and policies are seeded and versioned; budgets follow Theorem 9.9 ($M=738$, $\epsilon=\alpha=0.05$). Four questions: *RQ1* does NRP_{sys} detect degradation monotonically across the 21-adversary-parameter study? *RQ2* does the deterministic checker keep integrity against an off-critical-path LLM judge (Theorem 9.4, Theorem 9.5)? *RQ3* do the structure predictions of Section 9—propagation order (chain < tree < star) and the collusion advantage—appear empirically? *RQ4* what does the adversarial regime cost? Per-study tables and grids are in the Supplementary Material.

10.1 RQ1: system-level degradation under adversarial knobs

The headline stimulus is the defense $\times$ adversary-context study: each per-agent defense, single-agent (the ASB/AgentDojo reducer, Theorem 9.1) and under multi-agent collusion and a compromised orchestrator (star, $N=5$, $\beta=0.3$). The full grid is in the Supplementary Material. The guardrail’s relative ASR_{sys} reduction over no-defense shrinks from single-agent (41.6%) to collusion (31.8%) to compromised orchestrator (27.6%). Under the single-agent predicate the guardrail cuts ASR_{sys} from 0.587 to 0.343; under the full system predicate ASR_{sys} saturates at 1.0 in both multi-agent columns because the star hub collapses availability rather than exfiltrating.

10.2 RQ2: integrity of the deterministic checker

Theorem 9.4 guarantees that equal traces yield equal C verdicts. A constructed message-reading stand-in flips on 80/80 compromised episodes while C is invariant on 80/80, but that 100% is true by construction and bounds nothing, so we also measure *real* judges. Over 40 clean and 40 injection-paired compromised traces, sampled 5 times each, every backbone agrees with C on 1.000 of clean verdicts—these are valid judges, not broken ones. Reading messages then costs them: Qwen2.5-3B and Qwen2.5-7B are pushed off the recorded state in 0.315 and 0.348 of injected calls, while Llama-3.2-1B resists all of them. The more damaging number needs no injection at all: Qwen2.5-7B returns a different verdict for the *identical* clean trace in 14/40 traces, so the equality premise of Theorem 9.4 simply does not hold for a sampled judge. Unparseable replies are counted as unusable, never guessed. Judge reliability is therefore not a binary “hijackable or not”: the two Qwen judges are both hijackable and irreproducible, and the one that resists

injection does so only as reliably as its own sampling. This is the empirical case for Theorem 9.4.

10.3 Two channels of system failure: a real-LLM defense study

The synthetic sweeps hold the agent fixed, so they cannot show what a defense does to a model that *would* have complied. We therefore rerun the defense contrast on four real backbones in three families (Qwen2.5-3B, Qwen2.5-7B, Llama-3.2-1B, Mistral-7B), over the four topologies at $N=5$, $M=30$ episodes per cell, with a prompt-level defense that marks untrusted data as untrusted. The payload is fictitious, so any successful exfiltration is ours. For the three locally run backbones we release every decision with the exact prompts sent, so the defense reaching the model is checkable from the artifact rather than taken on trust; the hosted Mistral run predates prompt logging, records the defense condition and raw outputs only, and we say so rather than imply symmetry. Every cell in Table 1 is regenerated from that release by script, and the re-run is deterministic: all 36 local cells reproduce bit-identically from their seeds. Backbone ids, decoding parameters, the verbatim system/user prompts and the defense transform, and the compliance-scoring rule are all in the Supplementary Material.

The result is not “the defense works” but a *two-channel* picture, and both halves matter for how a system-level metric should be read.

When the model complies, the action channel alone fails the system. Undefended, Llama-3.2-1B and Mistral-7B comply on every attack cell, and ASR_{sys} is then 1.000 on every topology, chain included—the unsafe action alone discharges the system predicate, so topology carries no signal. A single-agent guardrail score would report these cells as safe.

When the defense induces refusal, the action channel is closed and topology becomes the whole story. On Qwen2.5-3B, Qwen2.5-7B, and Mistral-7B the defense yields 0/1075 compliant decisions (1075 pooled decisions per backbone, Wilson 95% upper bound 0.0036) and $\text{ASR}_{\text{act}}=0$ throughout—yet ASR_{sys} stays high and now discriminates sharply by topology, reaching 1.000 on tree and star under collusion (Table 1 and fig. 1). The residual failures are availability sabotage, not exfiltration: with the unsafe action blocked, the system predicate is still discharged by the deterministic availability path of Theorem 9.8, whose hub-critical topologies (tree, star) have no path to the sink. This is Theorem 9.7 observed with the action channel closed, and it is the reason a defense measured on the action channel cannot certify the system.

And the defense is model-dependent in a way that is easy to miss. Llama-3.2-1B ignores the defense entirely: 1075/1075 compliant decisions, $\text{ASR}_{\text{act}}=1$, $\text{ASR}_{\text{sys}}=1$ everywhere. Three of four backbones obey it, one does not, and the non-obeying case is exactly the small model that also complied when undefended. A defense’s measured effectiveness is therefore a property of the model as much as of the defense, and a study that reports an average over backbones can hide this.

Qwen2.5-3B, Qwen2.5-7B, and Mistral-7B return *bit-identical* ASR_{sys} in all twelve cells; once compliance is 0 the action channel is closed, so the system predicate follows topology and propagation

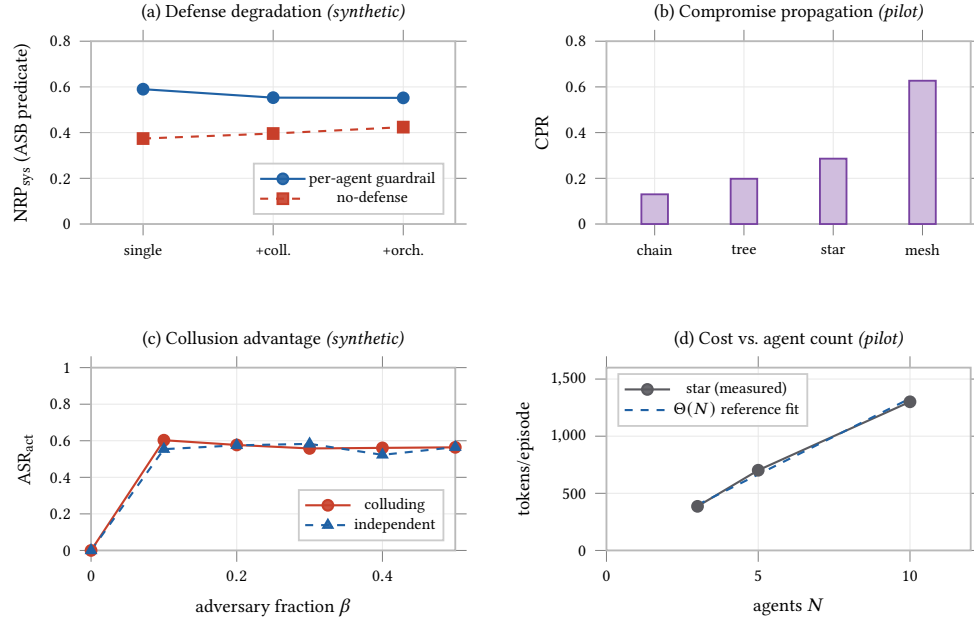


Figure 1: Measured trends. (a), (c): transparent synthetic model ($M=738$, $\epsilon=\alpha=0.05$, provenance-stamped). (b), (d): PILOT measurements, real backbones, 12 configs, 100 episodes/config. (a) The guardrail’s NRP_{sys} advantage over no-defense shrinks across single-agent, +collusion (+coll.), and +compromised-orchestrator (+orch.) (grid in the Supplementary Material). (b) CPR orders chain < tree < star < mesh (Section 9.3) and is identical across all backbones—a deterministic structural functional of the topology. (c) Collusion advantage in the unsafe-action channel is clearest at low β . (d) Real token cost grows $\approx$ linearly in N ; mesh and $N \in \{20, 50\}$ are deferred.

Table 1: Real-LLM defense study, prompt-level defense applied, $N=5$, $M=30$ episodes per cell, fictitious payload. Compliance is pooled over all 12 attack cells (1075 decisions, Wilson 95% upper bound 0.0036 where 0 is shown). ASR_{sys} is the system predicate under collusion. Where the defense holds, $\text{ASR}_{\text{act}}=0$ in every cell yet ASR_{sys} remains 1.000 on tree and star; Llama-3.2-1B does not obey the defense and fails on every topology. Table generated from the run summaries by experiments/make_defense_table.py.

Backbone	action channel	ASR_{sys} by topology (colluding)			
	compliance	chain	tree	star	mesh
Qwen2.5-3B	0.00	0.00	1.00	1.00	0.43
Qwen2.5-7B ^a	0.00	0.00	1.00	1.00	0.43
Llama-3.2-1B	1.00	1.00	1.00	1.00	1.00
Mistral-7B ^b	0.00	0.00	1.00	1.00	0.43

^a 4-bit quantized. ^b hosted, $M=30$. Episode-level 0.00 and 1.00 are 95% intervals $[0, 0.10]$ and $[0.90, 1]$; 0.43 is $[0.26, 0.62]$.

only. The compliance split (1 vs. 0) moves the collusion advantage from 0.00 to 0.33; see Supplementary Material.

10.4 RQ3: topology and collusion structure

Section 9.3 predicts chain < tree < star. The synthetic study confirms the order and the hub effect; Table 1 shows the same pattern

on real backbones. Under the system predicate, collusion saturates at $\text{ASR}_{\text{sys}}=1.000$ for all $\beta \geq 0.1$, with star saturation reflecting availability collapse. Per-backbone and per- β tables are in the Supplementary Material.

10.5 RQ4: cost

Cost is dominated by the episode budget of Theorem 9.9: a $G=21$ -configuration sweep with target gap Δ costs $\Theta(GN\Delta^{-2} \ln(G/\alpha))$ agent invocations (Theorem 9.10). Measured token/latency-versus- N and the full cost model are in the Supplementary Material.

11 Discussion, Limitations, and Ethics

Integrity: deterministic scoring prevents manipulation but requires well-specified predicates (Theorem 9.3). **Validity:** main gap is emulation fidelity ($\kappa \approx 0.48$); the emulated-versus-real-tools ablation is unmeasured; the deterministic-versus-judge ablation is reported. **Ethics:** only sandboxed fictitious payloads with bounded adversaries (Section 5) are used.

12 Conclusion

We presented MASGYM, a co-evolving red/blue security gym for multi-agent LLM systems. A deterministic scoring path isolates measurement from semantic manipulation. Topology governs spread under forced coordination and propagation. On real backbones, a prompt-level defense gave action-channel compliance of 0/1075

decisions but left $ASR_{sys}=1.000$ on tree and star, showing availability sabotage is topology-determined. MASGYM is a replicable instrument for system-level evaluation of multi-agent collectives.

References

- Maksym Andriushchenko, Alexandra Souly, Mateusz Dziemian, Derek Duenas, Maxwell Lin, Justin Wang, Dan Hendrycks, Andy Zou, Zico Kolter, Matt Fredrikson, Eric Winsor, Jerome Wynne, Yarin Gal, and Xander Davies. 2025. AgentHarm: A Benchmark for Measuring Harmfulness of LLM Agents. In *The Thirtieth International Conference on Learning Representations (ICLR)*. arXiv:2410.09024.
- Yoav Benjamini and Yoel Hochberg. 1995. Controlling the False Discovery Rate: A Practical and Powerful Approach to Multiple Testing. *Journal of the Royal Statistical Society: Series B (Methodological)* 57, 1 (1995), 289–300. doi:10.1111/j.2517-6161.1995.tb02031.x
- Peva Blanchard, El Mahdi El Mhamdi, Rachid Guerraoui, and Julien Stainer. 2017. Machine Learning with Adversaries: Byzantine Tolerant Gradient Descent. In *Advances in Neural Information Processing Systems 30 (NIPS)*.
- Stéphane Boucheron, Gábor Lugosi, and Pascal Massart. 2013. *Concentration Inequalities: A Nonasymptotic Theory of Independence*. Oxford University Press. doi:10.1093/acprof:oso/9780199535255.001.0001
- Greg Brockman, Vicki Cheung, Ludwig Pettersson, Jonas Schneider, John Schulman, Jie Tang, and Wojciech Zaremba. 2016. OpenAI Gym. *arXiv preprint arXiv:1606.01540* (2016).
- Mert Cemri, Melissa Z. Pan, Shuyi Yang, Lakshya A. Agrawal, Bhavya Chopra, Rishabh Tiwari, Kurt Keutzer, Aditya Parameswaran, Dan Klein, Kannan Ramchandran, Matei Zaharia, Joseph E. Gonzalez, and Ion Stoica. 2025. Why Do Multi-Agent LLM Systems Fail? *arXiv preprint arXiv:2503.13657* (2025). MAST failure taxonomy for multi-agent LLM systems.
- J. Chalupa, P. L. Leath, and G. R. Reich. 1979. Bootstrap Percolation on a Bethe Lattice. *Journal of Physics C: Solid State Physics* 12, 1 (1979), L31–L35. doi:10.1088/0022-3719/12/1/008
- Zhaorun Chen, Zhen Xiang, Chaowei Xiao, Dawn Song, and Bo Li. 2024. Agent-Poison: Red-teaming LLM Agents via Poisoning Memory or Knowledge Bases. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024)*. 130185–130213. arXiv:2407.12784.
- Jacob Cohen. 1960. A Coefficient of Agreement for Nominal Scales. *Educational and Psychological Measurement* 20, 1 (1960), 37–46. doi:10.1177/001316446002000104
- Stav Cohen, Ron Bitton, and Ben Nassi. 2024. Here Comes The AI Worm: Unleashing Zero-click Worms that Target GenAI-Powered Applications. *arXiv preprint arXiv:2403.02817* (2024).
- Edoardo DeBenedetti, Jie Zhang, Mislav Balunović, Luca Beurer-Kellner, Marc Fischer, and Florian Tramèr. 2024. AgentDojo: A Dynamic Environment to Evaluate Prompt Injection Attacks and Defenses for LLM Agents. In *Advances in Neural Information Processing Systems 37 (NeurIPS 2024), Datasets and Benchmarks Track*. doi:10.52202/079017-2636 deterministic state-based security() checks; arXiv:2406.13352.
- Yilun Du, Shuang Li, Antonio Torralba, Joshua B. Tenenbaum, and Igor Mordatch. 2023. Improving Factuality and Reasoning in Language Models through Multiagent Debate. *arXiv preprint arXiv:2305.14325* (2023).
- Joseph L. Fleiss. 1971. Measuring Nominal Scale Agreement Among Many Raters. *Psychological Bulletin* 76, 5 (1971), 378–382. doi:10.1037/h0031619
- Adam Fournay, Gagan Bansal, Hussein Mozannar, Cheng Tan, Eduardo Salinas, Erkang Zhu, et al. 2024. Magentic-One: A Generalist Multi-Agent System for Solving Complex Tasks. *arXiv preprint arXiv:2411.04468* (2024).
- Mark Granovetter. 1978. Threshold Models of Collective Behavior. *Amer. J. Sociology* 83, 6 (1978), 1420–1443. doi:10.1086/226707
- Xiangming Gu, Xiaosen Zheng, Tianyu Pang, Chao Du, Qian Liu, Ye Wang, Jing Jiang, and Min Lin. 2024. Agent Smith: A Single Image Can Jailbreak One Million Multimodal LLM Agents Exponentially Fast. In *Proceedings of the 41st International Conference on Machine Learning (ICML)*. arXiv:2402.08567.
- Lewis Hammond, Alan Chan, Jesse Clifton, Jason Hoelscher-Obermaier, Akbir Khan, Euan McLean, Chandler Smith, Wolfram Barfuss, Jakob Foerster, et al. 2025. *Multi-Agent Risks from Advanced AI*. Technical Report Technical Report #1. Cooperative AI Foundation. doi:10.48550/arXiv.2502.14143 arXiv:2502.14143.
- Feng He, Tianqing Zhu, Dayong Ye, Bo Liu, Wanlei Zhou, and Philip S. Yu. 2025. The Emerged Security and Privacy of LLM Agent: A Survey with Case Studies. *Comput. Surveys* (2025). doi:10.1145/3773080 arXiv:2407.19354.
- Wassily Hoeffding. 1963. Probability Inequalities for Sums of Bounded Random Variables. *J. Amer. Statist. Assoc.* 58, 301 (1963), 13–30. doi:10.1080/01621459.1963.10500830
- Sirui Hong, Mingchen Zhu, Jiaqi Chen, Xiawu Zheng, Yuheng Cheng, Ceyao Zhang, Jinlin Wang, Zili Wang, Steven Ka Shing Yau, Zijuan Lin, Liyang Zhou, Chenyu Ran, Lingfeng Xiao, Chenglin Wu, and Jürgen Schmidhuber. 2023. MetaGPT: Meta Programming for a Multi-Agent Collaborative Framework. *arXiv preprint arXiv:2308.00352* (2023).
- Wenyue Hua, Xianjun Yang, Mingyu Jin, Zelong Li, Wei Cheng, Ruixiang Tang, and Yongfeng Zhang. 2024. TrustAgent: Towards Safe and Trustworthy LLM-based Agents. In *Findings of the Association for Computational Linguistics: EMNLP 2024*. arXiv:2402.01586.
- Hakan Inan, Kartikeya Upasani, Jianfeng Chi, Rashi Rungta, Krithika Iyer, Yuning Mao, Michael Tontchev, Qing Hu, Brian Fuller, Davide Testuggine, and Madian Khabba. 2023. Llama Guard: LLM-based Input-Output Safeguard for Human-AI Conversations. *arXiv preprint arXiv:2312.06674* (2023).
- David Kempe, Jon Kleinberg, and Éva Tardos. 2003. Maximizing the Spread of Influence through a Social Network. In *Proceedings of the Ninth ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD '03)*. ACM, 137–146. doi:10.1145/956750.956769
- Leslie Lamport, Robert Shostak, and Marshall Pease. 1982. The Byzantine Generals Problem. *ACM Transactions on Programming Languages and Systems* 4, 3 (1982), 382–401. doi:10.1145/357172.357176
- Donghyun Lee and Mo Tiwari. 2024. Prompt Infection: LLM-to-LLM Prompt Injection within Multi-Agent Systems. *arXiv preprint arXiv:2410.07283* (2024).
- Erich L. Lehmann and Joseph P. Romano. 2005. *Testing Statistical Hypotheses* (3rd ed.). Springer. doi:10.1007/0-387-27605-X
- Joel Z. Leibo, Edgar Duéñez-Guzmán, Alexander Sasha Vezhnevets, John P. Agapiou, Peter Sunehag, Raphael Koster, Jayd Matyas, Charles Beattie, Igor Mordatch, and Thore Graepel. 2021. Scalable Evaluation of Multi-Agent Reinforcement Learning with Melting Pot. In *Proceedings of the 38th International Conference on Machine Learning (ICML) (PMLR, Vol. 139)*. 6187–6199. arXiv:2107.06857.
- Guohao Li, Hasan Abed Al Kader Hammoud, Hani Itani, Dmitrii Khizbullin, and Bernard Ghanem. 2023. CAMEL: Communicative Agents for “Mind” Exploration of Large Language Model Society. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2303.17760.
- Yuanchun Li, Hao Wen, Weijun Wang, Xiangyu Li, Yizhen Yuan, Guohong Liu, Jiacheng Liu, Wenxing Xu, Xiang Wang, Yi Sun, Rui Kong, Yile Wang, Hanfei Geng, Jian Luan, Xuefeng Jin, Zilong Ye, Guanqing Xiong, Fan Zhang, Xiang Li, Mengwei Xu, Zhijun Li, Peng Li, Yang Liu, Ya-Qin Zhang, and Yunxin Liu. 2024. Personal LLM Agents: Insights and Survey about the Capability, Efficiency and Security. *arXiv preprint arXiv:2401.05459* (2024).
- Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024. Formalizing and Benchmarking Prompt Injection Attacks and Defenses. In *33rd USENIX Security Symposium (USENIX Security)*. arXiv:2310.12815.
- Mantas Mazeika, Long Phan, Xu Wang Yin, Andy Zou, Zifan Wang, Norman Mu, Elham Sakhae, Nathaniel Li, Steven Basart, Bo Li, David Forsyth, and Dan Hendrycks. 2024. HarmBench: A Standardized Evaluation Framework for Automated Red Teaming and Robust Refusal. In *Proceedings of the 41st International Conference on Machine Learning (ICML) (PMLR, Vol. 235)*. 35181–35224. arXiv:2402.04249.
- Colin McDiarmid. 1989. On the Method of Bounded Differences. In *Surveys in Combinatorics, 1989*. London Mathematical Society Lecture Note Series, Vol. 141. Cambridge University Press, 148–188. doi:10.1017/CBO9781107359949.008
- Jerzy Neyman and Egon S. Pearson. 1933. On the Problem of the Most Efficient Tests of Statistical Hypotheses. *Philosophical Transactions of the Royal Society of London. Series A* 231, 694–706 (1933), 289–337. doi:10.1098/rsta.1933.0009
- Marshall Pease, Robert Shostak, and Leslie Lamport. 1980. Reaching Agreement in the Presence of Faults. *J. ACM* 27, 2 (1980), 228–234. doi:10.1145/322186.322188
- Ethan Perez, Saffron Huang, Francis Song, Trevor Cai, Roman Ring, John Aslanides, Amelia Glaese, Nat McAleese, and Geoffrey Irving. 2022. Red Teaming Language Models with Language Models. In *Proceedings of the 2022 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 3419–3448. doi:10.18653/v1/2022.emnlp-main.225 arXiv:2202.03286.
- Chen Qian, Wei Liu, Hongzhang Liu, Nuo Chen, Yufan Dang, Jiahao Li, Cheng Yang, Weize Chen, Yusheng Su, Xin Cong, Juyuan Xu, Dahai Li, Zhiyuan Liu, and Maosong Sun. 2024. ChatDev: Communicative Agents for Software Development. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (ACL)*. arXiv:2307.07924.
- Traian Rebedea, Razvan Dinu, Magesh Sreedhar, Christopher Parisien, and Jonathan Cohen. 2023. NeMo Guardrails: A Toolkit for Controllable and Safe LLM Applications with Programmable Rails. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing: System Demonstrations (EMNLP)*. arXiv:2310.10501.
- Yanguan Ruan, Honghua Dong, Andrew Wang, Silviu Pitis, Yongchao Zhou, Jimmy Ba, Yann Dubois, Chris J. Maddison, and Tatsunori Hashimoto. 2024. Identifying the Risks of LM Agents with an LM-Emulated Sandbox. In *The Twelfth International Conference on Learning Representations (ICLR)*. ToolEmu; arXiv:2309.15817.
- Donald B. Rubin. 1974. Estimating Causal Effects of Treatments in Randomized and Nonrandomized Studies. *Journal of Educational Psychology* 66 (1974), 688–701. doi:10.1037/h0037350
- Christian Schroeder de Witt, Klaudia Krawiecka, Igor Krawczuk, Ben Hagag, William L. Anderson, Peter Belcak, et al. 2025. Open Challenges in Multi-Agent Security: Towards Secure Systems of Interacting AI Agents. *arXiv preprint arXiv:2505.02077* (2025). doi:10.48550/arXiv.2505.02077 MASEC; Sec. 4.7 states no multi-agent security benchmark exists.
- J. K. Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, Niall Williams, Yashas Lokesh, and Praveen Ravi. 2021. PettingZoo: Gym for Multi-Agent Reinforcement Learning. In *Advances in Neural Information Processing*

- Systems 34 (NeurIPS)*. arXiv:2009.14471.
- Yu Tian, Xiao Yang, Jingyuan Zhang, Yinpeng Dong, and Hang Su. 2023. Evil Geniuses: Delving into the Safety of LLM-based Agents. *arXiv preprint arXiv:2311.11855* (2023).
- Boxin Wang, Weixin Chen, Hengzhi Pei, Chulin Xie, Mintong Kang, Chenhui Zhang, Chejian Xu, Zidi Xiong, Ritik Dutta, Rylan Schaeffer, Sang T. Truong, Simran Arora, Mantas Mazeika, Dan Hendrycks, Zinan Lin, Yu Cheng, Sanmi Koyejo, Dawn Song, and Bo Li. 2023a. DecodingTrust: A Comprehensive Assessment of Trustworthiness in GPT Models. In *Advances in Neural Information Processing Systems 36 (NeurIPS 2023)*, *Datasets and Benchmarks Track*. arXiv:2306.11698.
- Peiyi Wang, Lei Li, Liang Chen, Zefan Cai, Dawei Zhu, Binghuai Lin, Yunbo Cao, Qi Liu, Tianyu Liu, and Zhifang Sui. 2023b. Large Language Models Are Not Fair Evaluators. *arXiv preprint arXiv:2305.17926* (2023).
- Shilong Wang, Guibin Zhang, Miao Yu, Guancheng Wan, Fanci Meng, Chongye Guo, Kun Wang, and Yang Wang. 2025. G-Safeguard: A Topology-Guided Security Lens and Treatment on LLM-based Multi-agent Systems. *arXiv preprint arXiv:2502.11127* (2025).
- Duncan J. Watts. 2002. A Simple Model of Global Cascades on Random Networks. *Proceedings of the National Academy of Sciences* 99, 9 (2002), 5766–5771. doi:10.1073/pnas.082090499
- Qingyun Wu, Gagan Bansal, Jieyu Zhang, Yiran Wu, Beibin Li, Erkang Zhu, Li Jiang, Xiaoyun Zhang, Shaokun Zhang, Jiale Liu, Ahmed Hassan Awadallah, Ryan W. White, Doug Burger, and Chi Wang. 2023. AutoGen: Enabling Next-Gen LLM Applications via Multi-Agent Conversation. *arXiv preprint arXiv:2308.08155* (2023).
- Wenkai Yang, Xiaohan Bi, Yankai Lin, Sishuo Chen, Jie Zhou, and Xu Sun. 2024. Watch Out for Your Agents! Investigating Backdoor Threats to LLM-Based Agents. In *Advances in Neural Information Processing Systems (NeurIPS)*. arXiv:2402.11208.
- Miao Yu, Shilong Wang, Guibin Zhang, Junyuan Mao, Chenlong Yin, Qijiong Liu, Qingsong Wen, Kun Wang, and Yang Wang. 2024. NetSafe: Exploring the Topological Safety of Multi-agent Networks. *arXiv preprint arXiv:2410.15686* (2024).
- Yifan Zeng, Yiran Wu, Xiao Zhang, Huazheng Wang, and Qingyun Wu. 2024. AutoDefense: Multi-Agent LLM Defense against Jailbreak Attacks. *arXiv preprint arXiv:2403.04783* (2024).
- Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. InjecAgent: Benchmarking Indirect Prompt Injections in Tool-Integrated Large Language Model Agents. In *Findings of the Association for Computational Linguistics: ACL 2024*. arXiv:2403.02691.
- Hanrong Zhang, Jingyuan Huang, Kai Mei, Yifei Yao, Zhenting Wang, Chenlu Zhan, Hongwei Wang, and Yongfeng Zhang. 2025. Agent Security Bench (ASB): Formalizing and Benchmarking Attacks and Defenses in LLM-based Agents. In *The Thirteenth International Conference on Learning Representations (ICLR)*. ASB; introduces NRP = PNA (1 - ASR); arXiv:2410.02644.
- Lianmin Zheng, Wei-Lin Chiang, Ying Sheng, Siyuan Zhuang, Zhanghao Wu, Yonghao Zhuang, Zi Lin, Zhuohan Li, Dacheng Li, Eric P. Xing, Hao Zhang, Joseph E. Gonzalez, and Ion Stoica. 2023. Judging LLM-as-a-Judge with MT-Bench and Chatbot Arena. In *Advances in Neural Information Processing Systems (NeurIPS)*, *Datasets and Benchmarks Track*. arXiv:2306.05685.
- Zhenhong Zhou, Zherui Li, Jie Zhang, Yuanhe Zhang, Kun Wang, Yang Liu, and Qing Guo. 2025. CORBA: Contagious Recursive Blocking Attacks on Multi-Agent Systems Based on Large Language Models. *arXiv preprint arXiv:2502.14529* (2025).